

Technical Project Documentation

Integrated Educational Weather Station (Arduino + Raspberry Pi + MySQL)

Project: Environmental STEM Monitoring

Level: High School / K-12 STEM

Architecture: Arduino (Acquisition) & Python/MySQL (Processing)

1. Project Overview

Description: This project consists of an educational weather station designed and programmed by high school students to foster scientific learning, environmental monitoring, and technological development.

The system collects real-time atmospheric and environmental data using an Arduino microcontroller for analog/digital sensor reading, along with a single-board computer (Raspberry Pi/Linux) for advanced computation, persistent storage in a MySQL database, and local CSV logging.

2. Objectives and Scope

- **Educational & STEM:** Demonstrate how low-cost embedded systems can be utilized for climate monitoring, scientific experimentation, and classroom research.
- **Redundancy & Robustness:** Implement fault tolerance for critical sensors (such as automatic failover switching between primary and backup DHT11 sensors).
- **Multivariable Measurement:** Outdoor and shaded monitoring of temperature, humidity, atmospheric pressure, wind speed, accumulated rainfall, and air quality (PPM).
- **Software Integration:** Serial communication protocol between the microcontroller and the Python server with data persistence in MySQL (ba1un_canan).

3. Hardware Architecture and Sensors

Component / Sensor	Interface / Pin	Measured Variable	Function / Remarks
Arduino Microcontroller	Serial USB (9600 baud)	Real-time acquisition	Reads field sensors, handles pulses and interrupts, sends CSV frame over Serial.
BME280 (I2C)	I2C (0x76 / Bus 1)	Temp (°C), Pressure (hPa), Humidity (%)	High-precision sensor located on both the Arduino module and the Raspberry Pi server.
DHT11 (x2)	Digital Pin 5 (Pri) / Pin 8 (Sec)	Temperature / Relative Humidity	Backup system with power reset controlled via transistors/relays (Pins 7 and 4).
Analog Anemometer	Analog Pin A3	Wind Speed (m/s or km/h)	Linear calibration equation applied: $vel = 0.78 \times II - 5.46$.
Tipping Bucket Rain Gauge	Digital Pin 3 (INT)	Accumulated Rainfall (mm)	Mapped via hardware interrupt (FALLING). Calibrated to 0.3 mm/pulse with debouncing (300 ms).
Rainfall Sensor	Analog Pin A2	Qualitative rain detection	Identifies rain onset (≥ 200) or definitive rain (≥ 600).
MQ-135	Analog Pin A1	Air Quality / Gas Concentration (PPM)	Gas concentration calculation compensated for ambient temperature and humidity.

4. Arduino Source Code (`estacion\_meteorologica.ino`)

Arduino C++ - Sensor Acquisition and Control

```
#include <MQ135.h>
#include <Adafruit_BME280.h>
#include <Wire.h>
#include <DHT.h>

// Pin Definitions
const int pinV = 7;
const int pinV2 = 4;
const int Ane = A3;
const int sensorA = A2;
const int pluv = 3;
#define PIN_MQ135 A1

#define DHTPIN 5
#define DHTPIN1 8
#define DHTTYPE DHT11

DHT dht(DHTPIN, DHTTYPE);
DHT dht1(DHTPIN1, DHTTYPE);
Adafruit_BME280 bme;
MQ135 mq135_sensor(PIN_MQ135);

// Rain Gauge Variables
volatile int contadorPluv = 0;
volatile unsigned long ultimoPulso = 0;
volatile unsigned long ultimoPulsoReal = 0;
const float mmPulso = 0.3;
const int umbralTiempo = 300; // Debounce threshold (ms)

bool advertencial = false;
```

```

bool advertencia2 = false;

void setup() {
  Serial.begin(9600);
  dht.begin();
  dht1.begin();
  Wire.begin();

  bme.begin(0x76);

  pinMode(pinV, OUTPUT);
  pinMode(pinV2, OUTPUT);
  pinMode(Anc, INPUT);
  pinMode(sensorA, INPUT);
  pinMode(pluv, INPUT_PULLUP);

  digitalWrite(pinV, HIGH);
  digitalWrite(pinV2, HIGH);

  attachInterrupt(digitalPinToInterrupt(pluv), contarLluvia, FALLING);
}

void contarLluvia() {
  unsigned long ahora = millis();
  if (ahora - ultimoPulso > umbralTiempo) {
    contadorPluv++;
    ultimoPulso = ahora;
    ultimoPulsoReal = ahora;
  }
}

void loop() {
  float lecturaA = analogRead(sensorA);

  noInterrupts();
  int pulsos = contadorPluv;
  unsigned long ultimo = ultimoPulsoReal;
  interrupts();

  if (millis() - ultimo > 30000UL) {
    noInterrupts();
    contadorPluv = 0;
    interrupts();
    pulsos = 0;
  }

  float mmTotal = pulsos * mmPulso;

  // DHT11 Backup System Handling
  float tem1 = dht.readTemperature();
  float hum1 = dht.readHumidity();

  if ((isnan(tem1) || isnan(hum1)) && !advertencia1) {
    advertencia1 = true;
    Serial.println("Resetting primary DHT11...");
    digitalWrite(pinV, LOW);
    delay(2000);
    digitalWrite(pinV, HIGH);
    delay(2000);
    tem1 = dht.readTemperature();
    hum1 = dht.readHumidity();
  } else if ((isnan(tem1) || isnan(hum1)) && advertencia1 && !advertencia2) {
    Serial.println("Switching to backup DHT11...");
    advertencia2 = true;
    digitalWrite(pinV2, HIGH);
    tem1 = dht1.readTemperature();
    hum1 = dht1.readHumidity();
  }
}

```

```

} else if (isnan(tem1) || isnan(hum1)) {
  Serial.println("DHT11 ISSUE: CHECK SENSOR");
} else {
  digitalWrite(pinV, HIGH);
  digitalWrite(pinV2, HIGH);
}

float temp = bme.readTemperature();
float pres = bme.readPressure() / 100.0;

int l1 = analogRead(Ane);
float vel = 0.78 * l1 - 5.46;
if (vel < 0) vel = 0; // Avoid negative values due to offset at rest

float ppm = mq135_sensor.getCorrectedPPM(temp, hum1);

// Serial CSV Output: Temp_BME, Temp_DHT, Pressure, Wind_Speed, Rainfall, Humidity_DHT, PPM
Serial.print(temp); Serial.print(",");
Serial.print(tem1); Serial.print(",");
Serial.print(pres); Serial.print(",");
Serial.print(vel); Serial.print(",");
Serial.print(mmTotal); Serial.print(",");
Serial.print(hum1); Serial.print(",");
Serial.println(ppm);

delay(5000);
}

```

5. Python Source Code (`programa\_6.py`)

Python 3 - Processing Client, CSV & MySQL

```

import time
from datetime import datetime
import math
import os
import serial
from smbus2 import SMBus
from bme280 import BME280
import mysql.connector

# =====
# METEOROLOGICAL CALCULATIONS
# =====

def punto_rocio(temp, hum):
    a = 17.27
    b = 237.7
    alpha = ((a * temp) / (b + temp)) + math.log(hum / 100.0)
    return (b * alpha) / (a - alpha)

def sensacion_termica(temp, hum):
    return (-8.784695 +
            1.61139411 * temp +
            2.338549 * hum -
            0.14611605 * temp * hum -
            0.012308094 * (temp**2) -
            0.016424828 * (hum**2) +
            0.002211732 * (temp**2) * hum +
            0.00072546 * temp * (hum**2) -
            0.00003582 * (temp**2) * (hum**2))

# Sensor Setup
bus = SMBus(1)

```

```

bme280 = BME280(i2c_dev=bus)

# Database Connection
db_pass = os.environ.get("DB_PASSWORD", "3and1D0*")
db = mysql.connector.connect(
    host="localhost",
    user="root",
    password=db_pass,
    database="balun_canan"
)
cursor = db.cursor()

# Arduino Serial Connection
try:
    ser = serial.Serial('/dev/ttyACM0', 9600, timeout=2)
    time.sleep(2)
except Exception as e:
    print(f"❌ Error opening Serial port: {e}")
    ser = None

def leer_arduino():
    if ser is None:
        return None, None, None, None, None, None, None

    try:
        line = ser.readline().decode('utf-8').strip()
        if line:
            valores = line.split(',')
            if len(valores) == 7:
                temp_A = float(valores[0])
                temp_ext_A = float(valores[1])
                presion_A = float(valores[2])
                anemometro = float(valores[3])
                precipitacion = float(valores[4])
                humedad_A = float(valores[5])
                ppm_A = float(valores[6])
                return temp_A, temp_ext_A, presion_A, anemometro, precipitacion, humedad_A, ppm_A
            else:
                print(f"⚠️ Incomplete frame received. Length: {len(valores)}")
    except Exception as e:
        print(f"❌ Serial reading error: {e}")

    return None, None, None, None, None, None, None

# Main Loop
print('Starting weather station data collection...')
contador = 0

with open('datos.csv', 'a', buffering=1) as f:
    while True:
        try:
            temperatura = bme280.get_temperature()
            presion = bme280.get_pressure()
            humedad = bme280.get_humidity()

            dew = punto_rocio(temperatura, humedad)
            heat = sensacion_termica(temperatura, humedad)

            temp_A, temp_ext_A, presion_A, anemometro, precipitacion, humedad_A, ppm_A = leer_arduino()

            now = datetime.now()
            fecha_hora = now.strftime("%Y-%m-%d %H:%M:%S")

            print(f"[{fecha_hora}] Local BME280: {temperatura:.2f}°C | {presion:.2f} hPa | {humedad:.2f}%")
            print(f"    → Dew Point: {dew:.2f}°C | Heat Index: {heat:.2f}°C")
            print(f"    → Arduino: T={temp_A}, TExt={temp_ext_A}, P={presion_A}, Wind={anemometro},
Rain={precipitacion}")

```

```

        if -40 < temperatura < 85 and 300 < presion < 1100:
            f.write(f"{fecha_hora},{temperatura:.2f},{presion:.2f},{humedad:.2f},{dew:.2f},{heat:.2f},"
                    f"{temp_A},{temp_ext_A},{presion_A},{anemometro},{precipitacion},{humedad_A},{ppm_A}"
                    "\n")

            db.ping(reconnect=True)
            sql = ("INSERT INTO Datos_Arduino "
                    "(fecha, temperatura, presion, humedad, punto_rocio, sensacion_termica, "
                    "temp_A, temp_ext_A, presion_A, anemometro, precipitacion, humedad_A, ppm_A) "
                    "VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s)")

            valores_db = (fecha_hora, temperatura, presion, humedad, dew, heat,
                           temp_A, temp_ext_A, presion_A, anemometro, precipitacion, humedad_A, ppm_A)

            cursor.execute(sql, valores_db)
            contador += 1
            if contador >= 10:
                db.commit()
                print("✅ Batch of 10 records saved to MySQL")
                contador = 0

    except Exception as e:
        print("❌ Main loop error:", e)

    time.sleep(60) # Suggested sampling interval: 60 seconds

```

6. Installation and Setup Guide

1. **Arduino Configuration:** Install the MQ135, Adafruit_BME280, and DHT libraries in the Arduino IDE and upload the .ino sketch.
2. **MySQL Database Creation:**

```

CREATE DATABASE balun_canan;
USE balun_canan;

CREATE TABLE Datos_Arduino (
    id INT AUTO_INCREMENT PRIMARY KEY,
    fecha DATETIME,
    temperatura FLOAT,
    presion FLOAT,
    humedad FLOAT,
    punto_rocio FLOAT,
    sensacion_termica FLOAT,
    temp_A FLOAT,
    temp_ext_A FLOAT,
    presion_A FLOAT,
    anemometro FLOAT,
    precipitacion FLOAT,
    humedad_A FLOAT,
    ppm_A FLOAT
);

```

3. **Python Service Execution:**

```

pip install smbus2 matrix-bme280 mysql-connector-python pyserial
python3 programa_6.py

```